

# RecycleVision

Garbage image classification using deep learning: from a CNN built from scratch to a fine-tuned ResNet50 deployed as a live web app.

---

**97.4%**

Test accuracy (ResNet50)

---

**0.968**

Macro F1 across 12 classes

---

**15,515**

Training images

---

**3**

Models compared

battery



biological



brown-glass



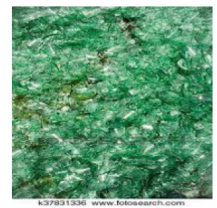
cardboard



clothes



green-glass



metal



paper



plastic



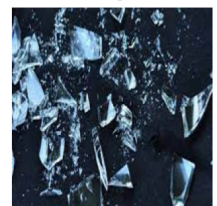
shoes



trash



white-glass



---

**Author** Mukundan Ramesh

**Date** September 2026

**Live app** [recyclevision-garbage-image-classification-using-deep-learning.streamlit.app](https://recyclevision-garbage-image-classification-using-deep-learning.streamlit.app)

**Source code** [github.com/mukundnramesh2605/RecycleVision--Garbage-Image-Classification-Using-Deep-Learning](https://github.com/mukundnramesh2605/RecycleVision--Garbage-Image-Classification-Using-Deep-Learning)

**Stack** Python 3.11 · PyTorch · torchvision · scikit-learn · Streamlit

# Contents

|                   |                                                            |                    |                                                           |
|-------------------|------------------------------------------------------------|--------------------|-----------------------------------------------------------|
| <a href="#">1</a> | <a href="#">Executive summary</a>                          | <a href="#">7</a>  | <a href="#">Real-world evaluation</a>                     |
| <a href="#">2</a> | <a href="#">Problem statement &amp; business use cases</a> | <a href="#">8</a>  | <a href="#">The RecycleVision web app</a>                 |
| <a href="#">3</a> | <a href="#">Dataset &amp; exploratory analysis</a>         | <a href="#">9</a>  | <a href="#">Live demo script</a>                          |
| <a href="#">4</a> | <a href="#">Methodology: data pipeline</a>                 | <a href="#">10</a> | <a href="#">Limitations, future work &amp; conclusion</a> |
| <a href="#">5</a> | <a href="#">Models &amp; training strategy</a>             | <a href="#">A</a>  | <a href="#">Appendix: reproducing the project</a>         |
| <a href="#">6</a> | <a href="#">Results on the test set</a>                    |                    |                                                           |

## 1 Executive summary

RecycleVision classifies a photo of a single waste item into one of **12 categories**: battery, biological, brown glass, cardboard, clothes, green glass, metal, paper, plastic, shoes, trash and white glass. The goal is to automate the first step of recycling, which is working out what an item is so it can go to the right stream.

Three models were trained and compared on an identical, stratified split of 15,515 images:

| MODEL                                          | TEST ACCURACY | MACRO F1     | PARAMETERS     |
|------------------------------------------------|---------------|--------------|----------------|
| Baseline CNN (from scratch)                    | 78.05%        | 0.735        | ~0.62 M        |
| MobileNetV2 (transfer learning)                | 95.96%        | 0.945        | ~2.2 M         |
| <b>ResNet50 (transfer learning) ✓ deployed</b> | <b>97.38%</b> | <b>0.968</b> | <b>~23.5 M</b> |

### Key takeaways

- **Transfer learning is decisive.** ImageNet-pretrained backbones added about 18–19 accuracy points over a CNN trained from scratch.
- **Two-phase training pays off.** Training only the new head first, then fine-tuning the whole network at a 10× lower learning rate, added about 2 points for both transfer models.
- **Class imbalance is handled.** With a class-weighted loss, even the smallest classes reach F1 above 0.90.
- **Real-world accuracy is lower.** On 120 web-scraped photos, accuracy drops to **77.5%**. The model is strong on clear single items and weaker on cluttered scenes.
- **It's live.** The best model runs in a Streamlit web app that anyone can try in a browser.

## 2 Problem statement & business use cases

---

Manual waste sorting is slow, costly and error-prone, and contamination (the wrong material in a recycling stream) lowers the value of everything collected. The aim of this project is to **build a deep learning model that classifies images of waste into material categories**, and to make it usable through a simple interface. It is a building block for automated sorting.

| USE CASE                   | HOW RECYCLEVISION HELPS                                                                   |
|----------------------------|-------------------------------------------------------------------------------------------|
| Smart recycling bins       | A camera in the bin lid identifies the item and opens the matching compartment.           |
| Municipal waste management | Pre-sorting on conveyor lines cuts manual labour and sorting time at recovery facilities. |
| Educational tools          | Instant visual feedback teaches households and schools to separate waste correctly.       |
| Environmental analytics    | Logging classifications over time shows waste composition and recycling trends.           |

### Project objectives

1. Prepare and explore a multi-class garbage image dataset, including its class imbalance.
2. Build a preprocessing and augmentation pipeline that every model shares.
3. Train a CNN from scratch as a reference point, then apply transfer learning (MobileNetV2, ResNet50).
4. Evaluate with accuracy, precision, recall, F1 and confusion matrices, and select the model with the best macro F1.
5. Check robustness on images from outside the dataset.
6. Deploy the chosen model as a Streamlit web application.

### Scope

Each input is a single image, and the output is one of 12 classes with a confidence score. Locating several items in one image (object detection) is out of scope and discussed in Section 10.

### 3 Dataset & exploratory analysis

Source: [Garbage Classification \(12 classes\)](#) on Kaggle, with **15,515 RGB images** in one folder per class.

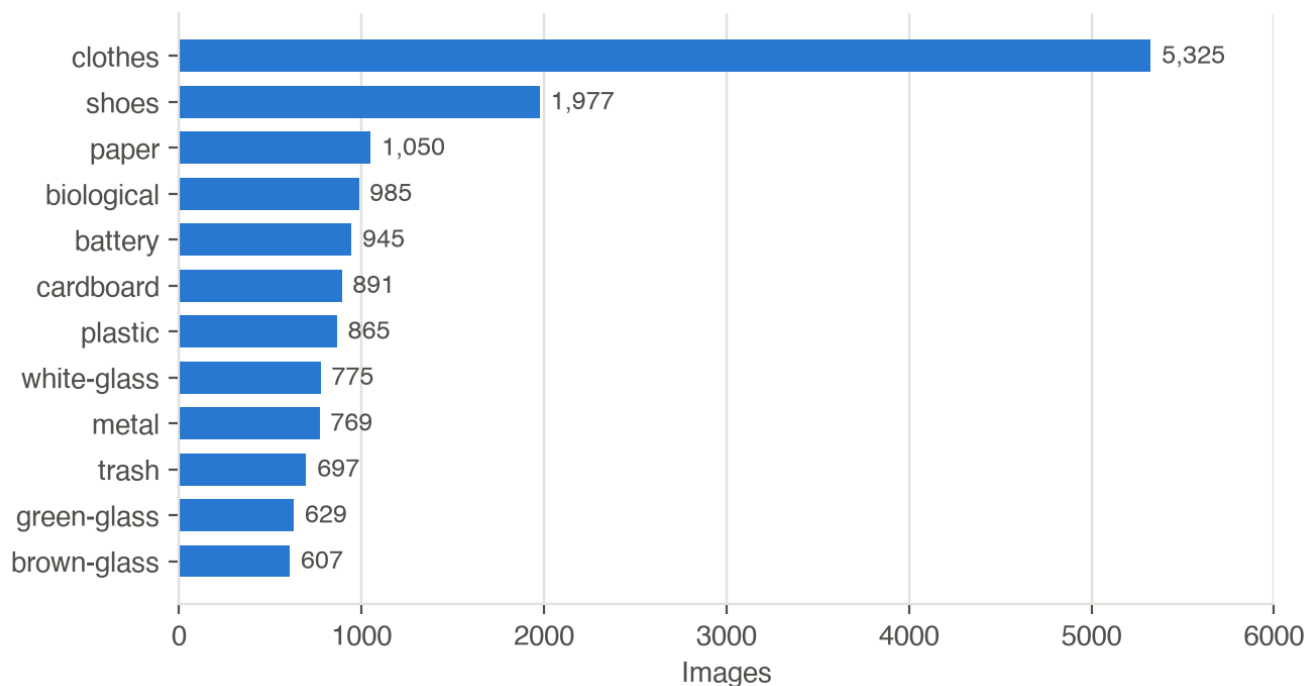


Figure 1. Images per class. `clothes` (5,325) has about 8.8× as many images as `brown-glass` (607).

#### Finding 1: strong class imbalance

`clothes` makes up 34% of the data and `shoes` another 13%. Trained naively, a model would favour these classes. This is handled with a class-weighted loss (Section 4).

#### Finding 2: variable image sizes

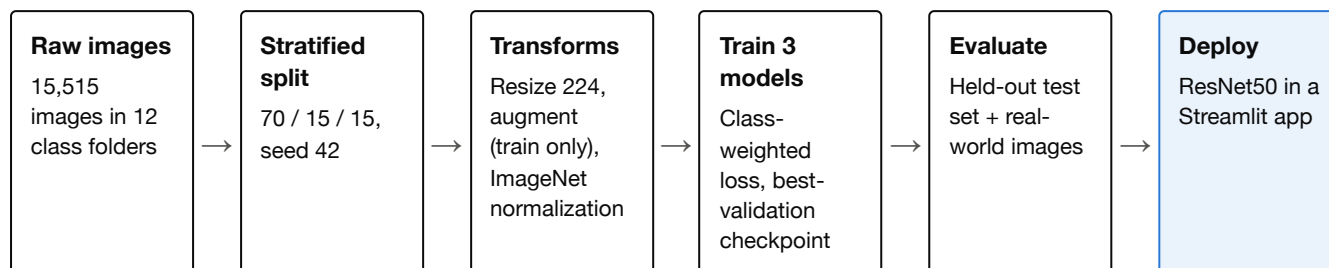
In a random sample of 200 images, widths ranged from 183 to 711 px and heights from 151 to 711 px, with 62 distinct sizes. Every image is therefore resized to a fixed 224×224 input.

#### Finding 3: visually similar classes

Three glass classes that differ mainly in colour, and several transparent or reflective materials (plastic, white glass, metal), are likely to be confused. The per-class results in Section 6 confirm this.

## 4 Methodology: data pipeline

All notebooks load data through one shared function, `get_data_loaders()`, so **every model trains and is tested on exactly the same images**. That makes the comparison fair.



### Train / validation / test split

| SPLIT      | SHARE | IMAGES  | BATCHES (32) | USED FOR                     |
|------------|-------|---------|--------------|------------------------------|
| Train      | 70%   | ~10,859 | 340          | Updating weights (augmented) |
| Validation | 15%   | ~2,328  | 73           | Picking the best epoch       |
| Test       | 15%   | 2,328   | 73           | Final, untouched evaluation  |

The split is stratified, so every class appears in the same proportion in each split. The test set is carved out first, then validation is taken from the remainder.

### Preprocessing & augmentation

| STEP                                            | TRAIN | VAL / TEST | WHY                                                    |
|-------------------------------------------------|-------|------------|--------------------------------------------------------|
| Resize to 224×224                               | ✓     | ✓          | Fixed input size expected by the CNN backbones         |
| Random horizontal flip                          | ✓     | —          | Orientation shouldn't change the class                 |
| Random rotation $\pm 15^\circ$                  | ✓     | —          | Robustness to tilted photos                            |
| Colour jitter (brightness, contrast $\pm 0.2$ ) | ✓     | —          | Robustness to lighting                                 |
| ImageNet mean/std normalization                 | ✓     | ✓          | Matches the statistics the pretrained backbones expect |

### Handling class imbalance

Class weights are computed with scikit-learn's `compute_class_weight("balanced")` from the **training labels only** (so no information leaks from the test set) and passed to `CrossEntropyLoss`. A mistake on a rare class therefore costs more:

| CLASS       | WEIGHT | CLASS       | WEIGHT | CLASS      | WEIGHT |
|-------------|--------|-------------|--------|------------|--------|
| brown-glass | 2.13   | metal       | 1.68   | biological | 1.31   |
| green-glass | 2.05   | white-glass | 1.67   | paper      | 1.23   |
| trash       | 1.85   | plastic     | 1.50   | shoes      | 0.65   |
| cardboard   | 1.45   | battery     | 1.37   | clothes    | 0.24   |

## 5 Models & training strategy

| MODEL        | ARCHITECTURE                                                                                                                     | ROLE                                                      |
|--------------|----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Baseline CNN | 3 × [Conv 3×3 → ReLU → MaxPool] with 32→64→128 filters, adaptive average pooling to 4×4, then Dense 256 → Dropout 0.3 → Dense 12 | Reference point: what a model learns from this data alone |
| MobileNetV2  | ImageNet-pretrained (IMAGENET1K_V1); final classifier replaced with Linear 1280 → 12                                             | Lightweight model suited to edge or mobile devices        |
| ResNet50     | ImageNet-pretrained (IMAGENET1K_V2); fully connected head replaced with Linear 2048 → 12                                         | High-capacity model aimed at top accuracy                 |

### Two-phase transfer learning

#### Phase 1: feature extraction (10 epochs, lr 1e-3)

The pretrained backbone is frozen and only the new 12-class head is trained. The randomly initialised head learns quickly without damaging the pretrained features.

#### Phase 2: fine-tuning (5 epochs, lr 1e-4)

The whole network is unfrozen and trained with a 10× lower learning rate, so the backbone adapts to garbage images without forgetting what it learned from ImageNet.

### Shared training setup

Adam optimizer · class-weighted cross-entropy · batch size 32 · trained on an Apple Silicon GPU (MPS). After every epoch, the weights with the **best validation accuracy** are kept, so the final model is the best checkpoint rather than the last one. The baseline was trained at lr 1e-3 for 2 warm-up epochs plus 15 epochs.

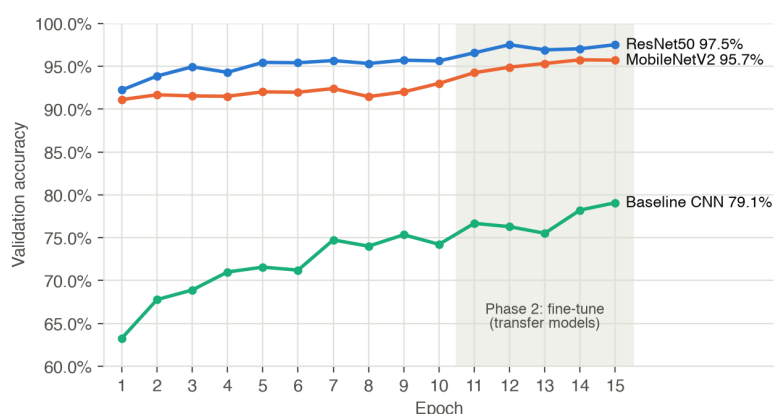


Figure 2. Validation accuracy per epoch. The shaded region is phase 2 (full fine-tuning) for the transfer models. Both transfer models start above 91% after one epoch, while the baseline is still climbing after 15.

| BEST VAL. ACC. | PHASE 1 | PHASE 2 |
|----------------|---------|---------|
| Baseline CNN   | 79.1%   | —       |
| MobileNetV2    | 93.0%   | 95.7%   |
| ResNet50       | 95.7%   | 97.5%   |

Fine-tuning added +2.7 points for MobileNetV2 and +1.8 for ResNet50. The baseline has a single training phase.

## 6 Results on the test set

All three saved models were reloaded and scored on the same **2,328 held-out test images**, which no model saw during training or checkpoint selection.

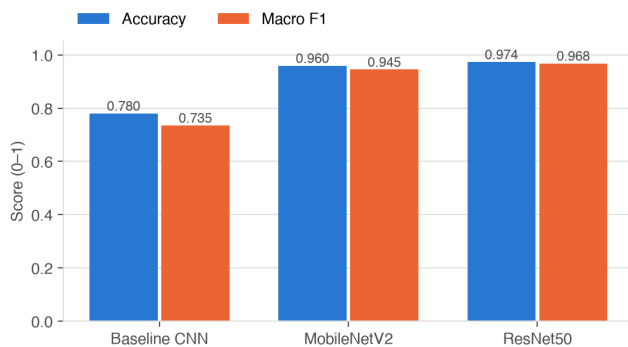


Figure 3. Test accuracy and macro F1 by model.

| MODEL        | ACC.  | MACRO F1 | WTD. F1 |
|--------------|-------|----------|---------|
| Baseline CNN | 0.781 | 0.735    | 0.780   |
| MobileNetV2  | 0.960 | 0.945    | 0.960   |
| ResNet50     | 0.974 | 0.968    | 0.974   |

### Why ResNet50 was selected

It has the highest **macro F1**, which weights all 12 classes equally and is the fairest metric on imbalanced data. MobileNetV2 comes within 1.4 accuracy points at about a tenth of the size, so it is the better choice for on-device deployment.

### ResNet50: per-class performance

| CLASS       | PREC. | RECALL | F1    | N     |
|-------------|-------|--------|-------|-------|
| battery     | 0.947 | 1.000  | 0.973 | 142   |
| biological  | 0.993 | 0.986  | 0.990 | 148   |
| brown-glass | 0.978 | 0.989  | 0.984 | 91    |
| cardboard   | 0.971 | 0.985  | 0.978 | 134   |
| clothes     | 0.996 | 0.980  | 0.988 | 799   |
| green-glass | 1.000 | 1.000  | 1.000 | 94    |
| metal       | 0.932 | 0.957  | 0.944 | 115   |
| paper       | 0.987 | 0.975  | 0.981 | 157   |
| plastic     | 0.914 | 0.900  | 0.907 | 130   |
| shoes       | 0.954 | 0.983  | 0.968 | 297   |
| trash       | 1.000 | 0.971  | 0.986 | 105   |
| white-glass | 0.930 | 0.914  | 0.922 | 116   |
| Macro avg   | 0.967 | 0.970  | 0.968 | 2,328 |

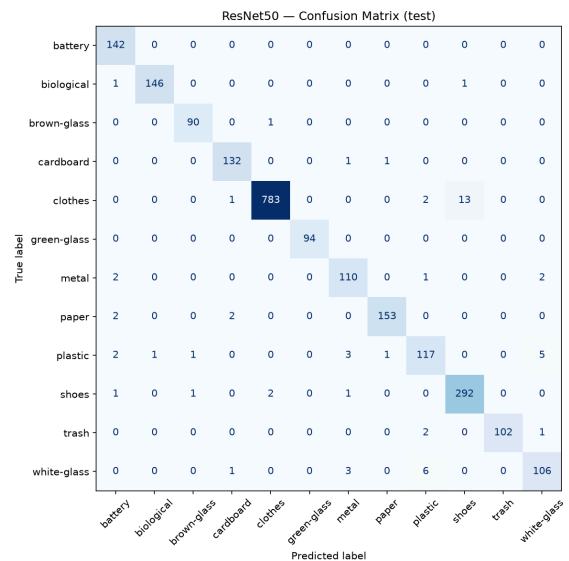


Figure 4. ResNet50 confusion matrix on the test set. Nearly all predictions fall on the diagonal.

**Strongest:** green-glass is perfect, and biological, clothes and trash are all at or above 0.986. **Weakest:** plastic (0.907), white-glass (0.922) and metal (0.944). These materials are transparent or reflective, and clear plastic bottles in particular resemble white glass. Class weighting worked: the rare classes score about as well as the dominant `clothes` class.

## 7 Real-world evaluation

Kaggle images are clean, well lit and usually show one object. To test how the model holds up outside that setting, 10 images per class (**120 in total**) were scraped from Bing image search using descriptive queries such as "rotten fruit vegetable" for biological and "metal can" for metal. The deployed ResNet50 was then scored on them with the same preprocessing as the test set.

| METRIC   | KAGGLE TEST SET | REAL-WORLD IMAGES |
|----------|-----------------|-------------------|
| Accuracy | 97.38%          | <b>77.50%</b>     |
| Macro F1 | 0.968           | <b>0.738</b>      |
| Images   | 2,328           | 120               |

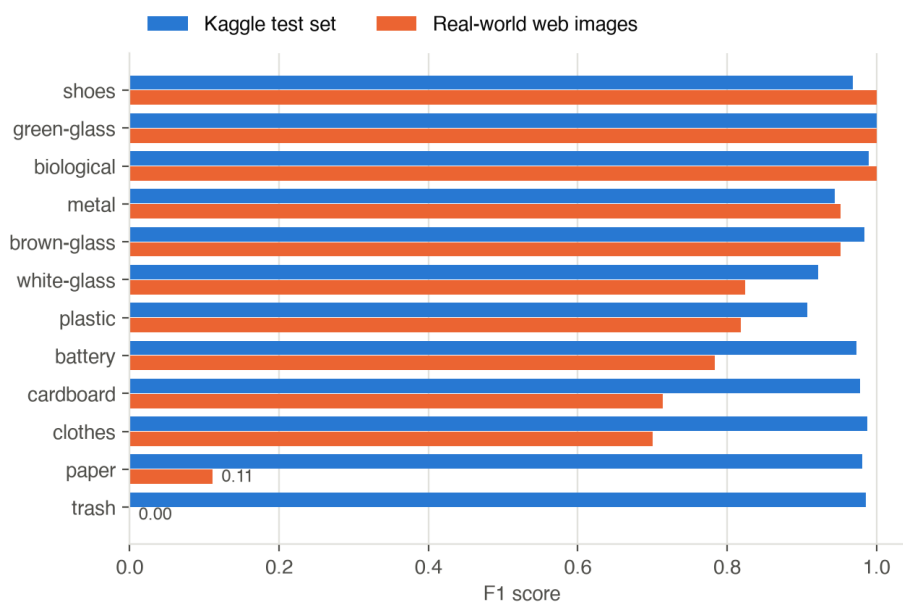


Figure 5. Per-class F1 on the Kaggle test set compared with real-world web images, sorted by real-world F1.

### What transfers well

Objects with a distinctive shape or texture keep their performance: **biological, green-glass and shoes stay at 1.000**, and metal and brown-glass stay above 0.95.

### What breaks

**trash (0.00)** and **paper (0.11)** collapse. The "trash" query returned whole scenes of bins and landfills, which look nothing like the single-item Kaggle trash class. **cardboard** becomes a catch-all wrong answer, and its precision falls to 0.556.

#### How to read this result

The drop has two causes: a genuine **domain gap** (training on clean single objects, testing on cluttered scenes) and a **data-collection artifact** (web queries that don't match the class definitions). The scraped labels were not manually verified, and 10 images per class is a small sample, so treat 77.5% as indicative rather than precise. The model is strong on clear single items and needs more work before it can handle mixed, cluttered waste.

## 8 The RecycleVision web app

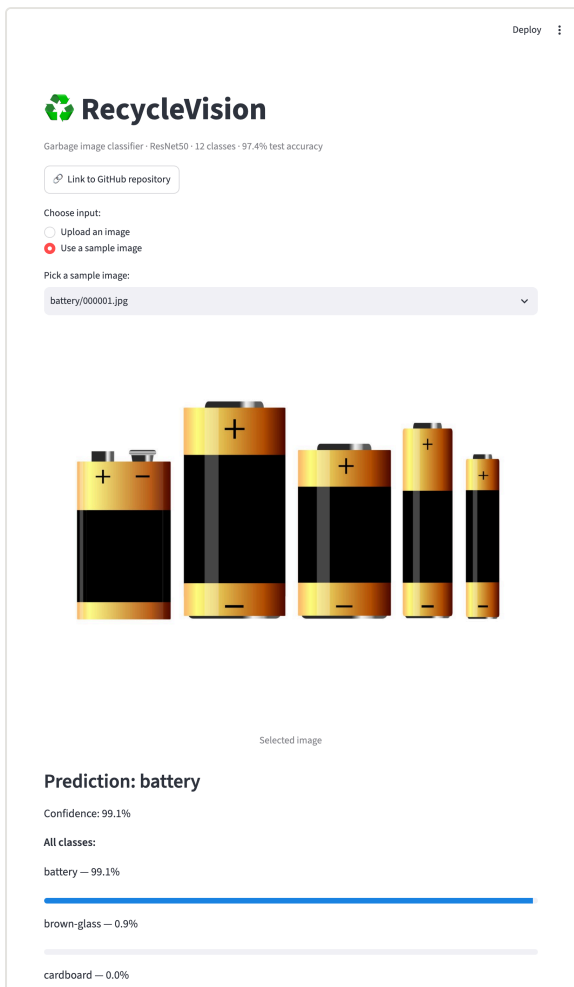


Figure 6. The app in sample-image mode, classifying a battery at 99.1% confidence. Captured from the running app.

Live at [recyclevision-garbage-image-classification-using-deep-learning.streamlit.app](https://recyclevision-garbage-image-classification-using-deep-learning.streamlit.app), hosted on Streamlit Community Cloud.

### Features

- **Two input modes:** upload your own JPG or PNG, or choose one of 120 bundled sample images from a dropdown, so no image of your own is needed.
- **Prediction with confidence:** the top class and its probability.
- **Full probability breakdown:** all 12 classes, ranked, each with a progress bar.
- **GitHub link button** in the header.

### How it works

1. The image is converted to RGB and resized to 224×224.
2. It is normalized with ImageNet mean and standard deviation, the same as at evaluation time.
3. ResNet50 produces 12 logits, and softmax turns them into probabilities.

### Engineering notes

- The model is loaded once and cached with `@st.cache_resource`.
- Inference runs on the **CPU**, so no GPU is needed.
- `requirements.txt` installs the CPU-only PyTorch build, which keeps the cloud deployment small.
- If the sample folder is empty, the app shows a warning instead of failing.

## 9 Live demo script

### Sample predictions to show

Actual outputs of the deployed ResNet50 on images in `sample_images/` (top prediction, then the next two).



BATTERY/000002.JPG  
✓ **battery >99.9%**  
metal 0.0% · paper 0.0%



GREEN-GLASS/000001.JPG  
✓ **green-glass >99.9%**  
brown-glass 0.0% · white-glass 0.0%



BIOLOGICAL/000001.JPG  
✓ **biological 99.0%**  
paper 0.7% · clothes 0.2%



SHOES/000001.JPG  
✓ **shoes 99.4%**  
clothes 0.6% · metal 0.0%



METAL/000001.JPG  
✓ **metal >99.9%**  
white-glass 0.0% · plastic 0.0%



PLASTIC/000001.JPG  
✓ **plastic 99.9%**  
brown-glass 0.1% · white-glass 0.0%



CARDBOARD/000001.JPG  
✓ **cardboard >99.9%**  
paper 0.0% · brown-glass 0.0%



PAPER/000001.JPG · TRUE: PAPER  
✗ **cardboard 59.3%**  
paper 23.9% · white-glass 7.8%

### Suggested walkthrough (about 5 minutes)

- 1 Frame the problem.** "Sorting waste by hand is slow and error-prone. RecycleVision tells you what an item is from a single photo, across 12 categories."
- 2 Open the live app** and point out the header: ResNet50, 12 classes, 97.4% test accuracy.
- 3 Easy win:** choose *Use a sample image* → `green-glass/000001.jpg`. The confidence is above 99.9%. Explain the ranked bars, where the runner-ups are the other glass colours, which makes sense.
- 4 Robustness:** try `biological/000001.jpg` or `shoes/000001.jpg`. These are messy web photos, not the training dataset, and the model is still about 99% confident.
- 5 Honest failure:** show `paper/000001.jpg`. The model predicts cardboard at 59% with paper second at 24%. Use it to lead into the real-world evaluation: 97.4% on clean test data, 77.5% on web images.
- 6 Upload mode:** if there's time, photograph an item in the room and upload it.
- 7 Close with the numbers:** the model comparison chart (Figure 3) and the future work in Section 10.

## 10 Limitations, future work & conclusion

### Limitations

- **Domain shift.** Training images mostly show one object on a plain background. Real bins are cluttered and poorly lit (Section 7).
- **Confusable materials.** Plastic, white glass and metal are the weakest classes even on clean data.
- **Single-label output.** One image gets one class, so a mixed pile can't be described.
- **Ambiguous class definitions.** "Trash" and "paper" mean different things in different contexts, as the web-image results showed.
- **Small real-world sample.** The 120 scraped images are unverified, so the real-world numbers are indicative only.

### Future work

| IDEA                                                                 | EXPECTED BENEFIT                                              |
|----------------------------------------------------------------------|---------------------------------------------------------------|
| Train on in-the-wild images (hard-negative mining, cluttered scenes) | Close the 97% → 77% domain gap                                |
| Object detection (for example YOLO) for mixed waste                  | Classify every item in a scene, not just one                  |
| EfficientNet or ConvNeXt backbones, LR scheduling, early stopping    | Higher accuracy, especially on plastic and white glass        |
| Grad-CAM heatmaps in the app                                         | Show users what the model looked at, which builds trust       |
| Top-3 view and a "not sure" state below a confidence threshold       | Safer behaviour on ambiguous inputs such as the paper example |
| Export MobileNetV2 to ONNX or TorchScript                            | On-device inference for smart bins                            |

### Conclusion

RecycleVision shows that **transfer learning turns a modest dataset into a highly accurate waste classifier**. A fine-tuned ResNet50 reaches 97.4% accuracy and 0.968 macro F1 across 12 imbalanced classes, well above the 78% of a CNN trained from scratch. It is deployed as a public web app that anyone can try. The real-world test gives a candid view of where it stands: reliable on clear single items, with a clear path (in-the-wild training data and object detection) toward the cluttered conditions of real smart-bin and sorting-line deployments.

### Tech stack

|                           |                                                         |                          |                                      |
|---------------------------|---------------------------------------------------------|--------------------------|--------------------------------------|
| <b>Language</b>           | Python 3.11                                             | <b>Visualisation</b>     | Matplotlib                           |
| <b>Deep learning</b>      | PyTorch, torchvision                                    | <b>App &amp; hosting</b> | Streamlit, Streamlit Community Cloud |
| <b>Data &amp; metrics</b> | NumPy, pandas, scikit-learn                             | <b>Data collection</b>   | icrawler (Bing image search)         |
| <b>Hardware</b>           | Apple Silicon GPU (MPS) for training; CPU for inference |                          |                                      |

# A Appendix: reproducing the project

## Repository layout

```
01_eda.ipynb           Exploratory data analysis
02_baseline_cnn.ipynb  CNN trained from scratch
03_transfer_learning.ipynb MobileNetV2 + ResNet50 (freeze → fine-tune)
04_evaluation.ipynb    Test metrics, confusion matrices, comparison
app.py                 Streamlit app (ResNet50)
download_real_world_dataset.py Bing scraper for real-world images
helper_functions/data_loading/data_utils.py get_data loaders()
models/                BaselineCNN class + trained weights (.pt)
reports/figures, reports/metrics Plots and JSON results
sample_images/, real_world_test/ 120 web images (10 per class)
```

## Run the app locally

```
git clone https://github.com/mukundnaramesh2605/RecycleVision--Garbage-Image-Classification-Using-Deep-Learning.git
cd RecycleVision--Garbage-Image-Classification-Using-Deep-Learning
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py # then open http://localhost:8501
```

## Retrain from scratch

1. Download the [Kaggle dataset](#) into `dataset/garbage_classification/`.
2. Install the notebook dependencies: `pip install torch torchvision scikit-learn matplotlib pandas jupyter`.
3. Run the notebooks in order (01 → 04) from the project root. The split is fixed with `seed=42`.

## Confusion matrices: baseline and MobileNetV2

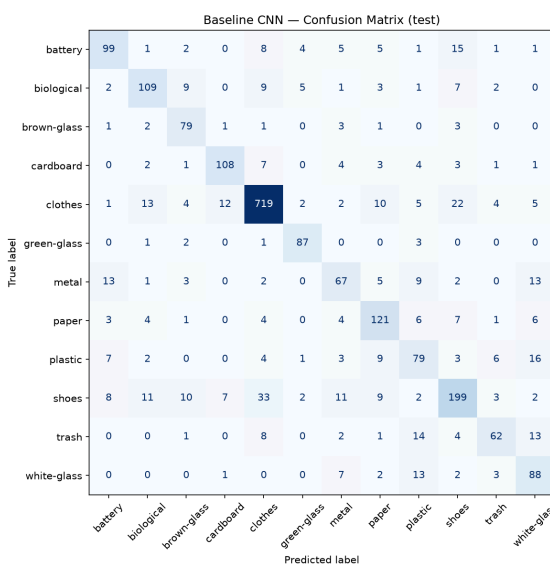


Figure 7. Baseline CNN (test accuracy 78.1%).

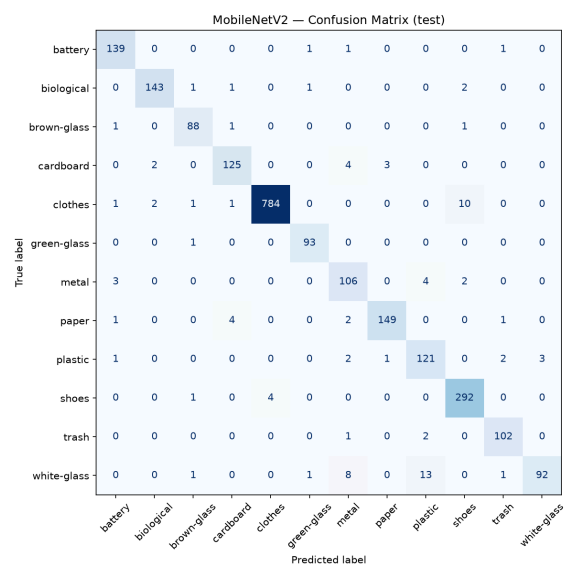


Figure 8. MobileNetV2 (test accuracy 96.0%).